

05.05.25

Tasks for now:

1. Clone Kep's Git +
2. Make Jonas a contributor +
3. Create two branches
4. Push my current code
5. Keep building for a week, then compare the effectiveness of encodings, choose the best one.

08.05.25

Let's check if the new structure / previous code has been pushed to git. **Done.**

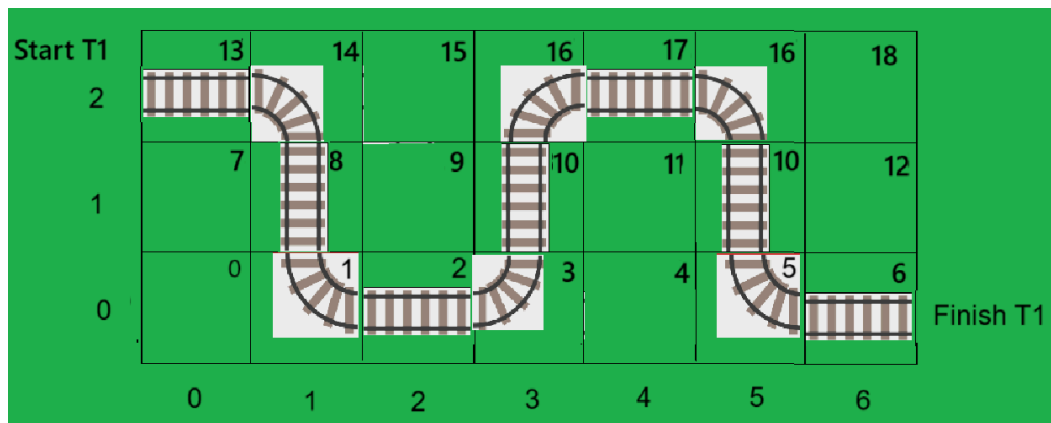
—

Now let's get back to calmly experimenting with our models - and push the daily work to git by the end of the day.

—

What next more complicated - but not too complicated - environment can I test my model on?

The last one was a Π -turn. Let's introduce a 'snake' - still one train deterministic environment, but with a more complex structure.



Let's describe it in the required input format:

```
cell((0,0), 0).  
cell((0,1), 72).
```

```

cell((0,2), 1025).
cell((0,3), 2064).
cell((0,4), 0).
cell((0,5), 72).
cell((0,6), 1025).
cell((1,0), 0).
cell((1,1), 32800).
cell((1,2), 0).
cell((1,3), 32800).
cell((1,4), 0).
cell((1,5), 32800).
cell((1,6), 0).
cell((2,0), 1025).
cell((2,1), 4608).
cell((2,2), 0).
cell((2,3), 16386).
cell((2,4), 1025).
cell((2,5), 4608).
cell((2,6), 0).

train(1).
start(1, (2, 0), 0, east).
end(1, (0, 6), 13).

```

Done. Let's plug it into our model and see what it offers.

It's unsatisfiable! Why? So, we may conclude that the problem is not in two trains, but in how the structure of the environment is processed.

H1: positional converter does wrong;

Let's display and examine the 'path' atoms.

Result: even the path production is impossible! Why? Let's switch off constraints one by one.

This constraint:

```

:- transition(TRAIN, MOMENT, _, TO_CELL),
   transition(TRAIN, MOMENT+1, FROM_CELL, _),
   TO_CELL != FROM_CELL.

```

Blocked all answers. Why?

Discovery: it broke down for all environments! How? Why? Where?

I don't understand why it doesn't work and it's very annoying, because it DID work.

H2: some of the constraints prevent any answers;

It only works for the Π -turn now. On the Γ -turn too! It's **very dependent on the hard constraint that is based on the arrival time**.

And there was a mistake in the input for the straight line - a simple coordinate confusion.

Conclusion: be *extremely* accurate with the input, since the model now is very sensitive to wrong arrival times / cell positions (the latter is as it must be!).

Oh, now it finally just runs again with all the previous very simple nets. Let's finally go to the snake.

There is some mess with arrival time encoding. Let's get the one that is given in the 'official' test environments. First, is time 0 or 1 based? Okay, let's assume it's zero based. Then anyway, the moment of time that is given in the atom that looks like this:

```
end(0, (16,24), 59) .
```

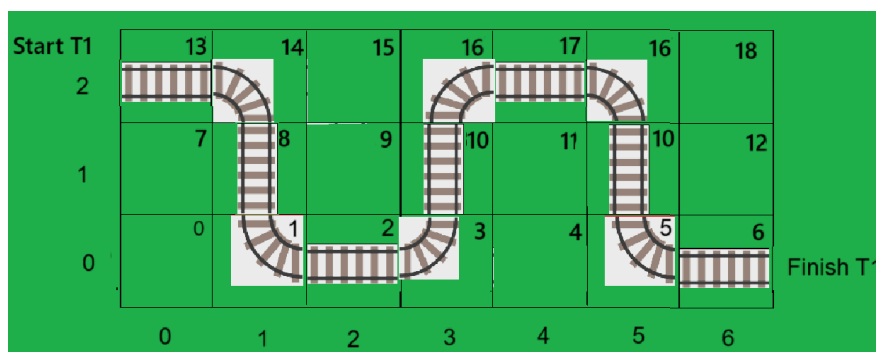
Denotes the time when the train must be in the final cell, not a 'transition to the final cell moment'. Let's fix this.

But, if the moment counter is 0-based, then my encoding is correct: to obtain the time of arrival from my encoding, I must simply add 1 to the final 'transition' atom, like this:

```
arrival_time(END_TIME+1) :- transition(END_TIME)
```

Let's check the output for the 'snake'.

```
transition(1,0,14,15) transition(1,1,15,8) transition(1,2,8,1)
transition(1,3,1,2) transition(1,4,2,3) transition(1,5,3,10)
transition(1,6,10,17) transition(1,7,17,18) transition(1,8,18,19)
transition(1,9,19,12) transition(1,10,12,5) transition(1,11,5,6)
```



Is everything shifted by 1 or what? Something is **very broken** here.

H1: my original encoding of the environment is incorrect;

...

H2: converter is incorrect;

- Could it be that it fails on non-square environments?

Let's check how it derives grid size:

```
grid_size(3,7)
```

No, it's correct. Let's check the 'path' atoms:

```
path(north,8,north,15) path(north,10,north,17) path(north,12,north,19)
path(south,8,south,1) path(south,10,south,3) path(south,12,south,5)
path(east,2,east,3) path(east,6,east,7) path(east,14,east,15)
path(east,18,east,19) path(west,2,west,1) path(west,6,west,5)
path(west,14,west,13) path(west,18,west,17) path(east,15,south,8)
path(east,19,south,12) path(north,15,west,14) path(north,19,west,18)
path(west,17,south,10) path(north,17,east,18) path(west,1,north,8)
path(west,5,north,12) path(south,1,east,2) path(south,5,east,6)
path(east,3,north,10) path(south,3,west,2)
```

There are some completely crazy paths here. Where exactly does it break down?

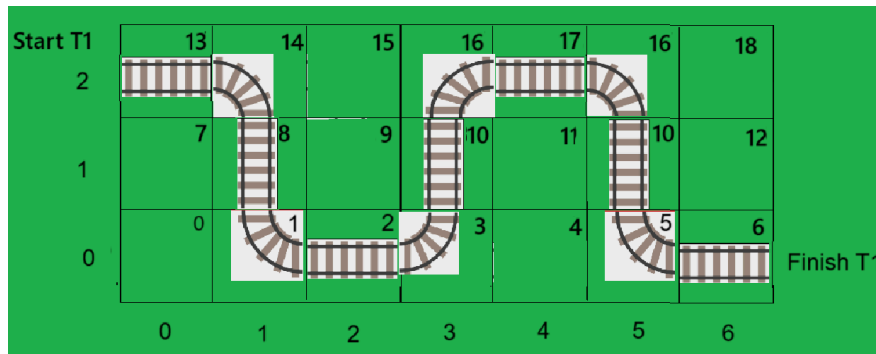
- Check derivations of 'path' atoms where it falters.
- ...
- ...

A great idea is to show 'cell_flat' atoms produced by the converter - the mistake might be there:

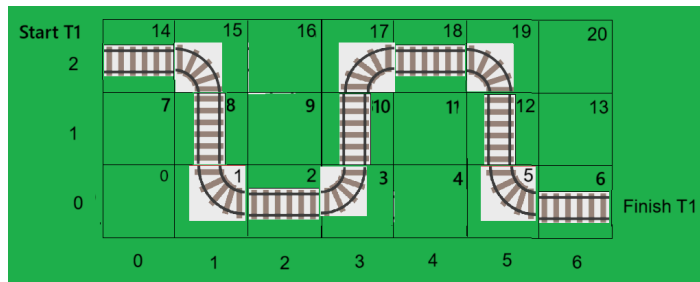
```

cell_flat(0,0) cell_flat(1,72) cell_flat(2,1025) cell_flat(3,2064)
cell_flat(4,0) cell_flat(5,72) cell_flat(6,1025) cell_flat(7,0)
cell_flat(8,32800) cell_flat(9,0) cell_flat(10,32800) cell_flat(11,0)
cell_flat(12,32800) cell_flat(13,0) cell_flat(14,1025)
cell_flat(15,4608) cell_flat(16,0) cell_flat(17,16386)
cell_flat(18,1025) cell_flat(19,4608) cell_flat(20,0)

```



The 21 cell output size is correct, but it's mixed up - most likely in my original input. Let's look at it: ... No! Let's first redraw it:



```

cell((0,0), 0).
cell((0,1), 72).
cell((0,2), 1025).
cell((0,3), 2064).
cell((0,4), 0).
cell((0,5), 72).
cell((0,6), 1025).
cell((1,0), 0).
cell((1,1), 32800).
cell((1,2), 0).
cell((1,3), 32800).
cell((1,4), 0).

```

```

cell((1,5), 32800).
cell((1,6), 0).
cell((2,0), 1025).
cell((2,1), 4608).
cell((2,2), 0).
cell((2,3), 16386).
cell((2,4), 1025).
cell((2,5), 4608).
cell((2,6), 0).

```

The initial encoding is correct.

H3: I simply misinterpreted the correct output due to the original mistake in the picture.

Let's run again to see only 'path' atoms and compare against the image:

```

path(north,8,north,15) path(north,10,north,17) path(north,12,north,19) path(south,8,south,1)
path(south,10,south,3) path(south,12,south,5) path(east,2,east,3) path(east,6,east,7) path(east,14,east,15)
path(east,18,east,19) path(west,2,west,1) path(west,6,west,5) path(west,14,west,13) path(west,18,west,17)
path(east,15,south,8) path(east,19,south,12) path(north,15,west,14) path(north,19,west,18)
path(west,17,south,10) path(north,17,east,18) path(west,1,north,8) path(west,5,north,12) path(south,1,east,2)
path(south,5,east,6) path(east,3,north,10) path(south,3,west,2)

```

Everything is correct, the two potentially problematic 'path' atoms are:

```
path(east,6,east,7)
path(west,14,west,13)
```

These transitions don't exist - the model 'thinks' is a possible transition due to type description, but for some reason neglects the condition for the accepting cell to exist. Let's check this.

I see the problem:

```
path(east, POSITION, east, POSITION + 1) :- cell_flat(POSITION, 1025),
cell_flat(POSITION + 1, TYPE).
```

My encoding assumes the existence of ANY acceptance cells, while it must be any *non-zero* cell. Let's fix this:

So, I simply added:

```
TYPE != 0
```

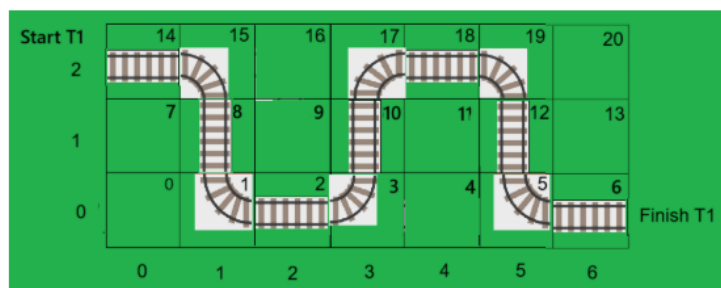
To every line that is responsible for remaking input into 'path' atoms. Let's test.

```
path(north,8,north,15) path(north,10,north,17) path(north,12,north,19) path(south,8,south,1)
path(south,10,south,3) path(south,12,south,5) path(east,2,east,3) path(east,14,east,15)
path(east,18,east,19) path(west,2,west,1) path(west,6,west,5) path(west,18,west,17)
path(east,15,south,8) path(east,19,south,12) path(north,15,west,14) path(north,19,west,18)
path(west,17,south,10) path(north,17,east,18) path(west,1,north,8) path(west,5,north,12)
path(south,1,east,2) path(south,5,east,6) path(east,3,north,10) path(south,3,west,2)
```

Now, only the correct path atoms are present. Let's test the final output in terms of 'transition' atoms:

```
transition(1,0,14,15) transition(1,1,15,8) transition(1,2,8,1) transition(1,3,1,2)
transition(1,4,2,3) transition(1,5,3,10) transition(1,6,10,17) transition(1,7,17,18)
transition(1,8,18,19) transition(1,9,19,12) transition(1,10,12,5) transition(1,11,5,6)
```

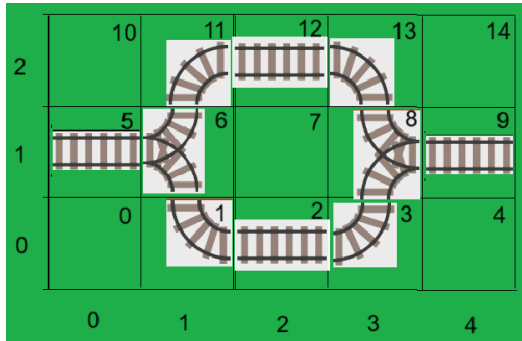
Let's check if it's the right path:



Yes, this is fully correct.

IDEA: non-deterministic environment for 1 train, but with equal arrival time in both branches.

Let's draw such one first:



And describe in standard input atoms:

```
cell((0,0), 0).
cell((0,1), 72).
cell((0,2), 1025).
cell((0,3), 2064)
cell((0,4), 0).
cell((1,0), 1025).
cell((1,1), 6672).
cell((1,2), 0).
```

```
cell((1,3), 16458).
cell((1,4), 1025).
cell((2,0), 0).
cell((2,1), 16386).
cell((2,2), 1025).
cell((2,3), 4608).
cell((2,4), 0).
```

Now, the problem is that two types of cells are not described in the current encoding. Let's do this - the description will differ from the one that exists for deterministic types, but will be based on it:

```
path(west, POSITION, south, POSITION - WIDTH) :- cell_flat(POSITION, 16386), grid_size(_,
WIDTH), cell_flat(POSITION - WIDTH, TYPE), TYPE != 0.
```

Everything I need is already in this table, perhaps:

https://docs.google.com/spreadsheets/d/1vVDptAVsx0L1_bZe3m-rT9NVDgNmR9R0W8ZQWghZTlw/edit?gid=0#gid=0

Let's find the missing track types in it: 6672, 16458.

6672: three options:

```
path(east, POSITION, south, POSITION - WIDTH) :- cell_flat(POSITION, 6672), grid_size(_,
WIDTH), cell_flat(POSITION - WIDTH, TYPE) != 0.
```

```
path(east, POSITION, north, POSITION + WIDTH) :- cell_flat(POSITION, 6672), grid_size(_,
WIDTH), cell_flat(POSITION + WIDTH, TYPE) != 0.
```

```
path((south;north), POSITION, west, POSITION - 1) :- cell_flat(POSITION, 6672),
cell_flat(POSITION + WIDTH, TYPE) != 0.
```

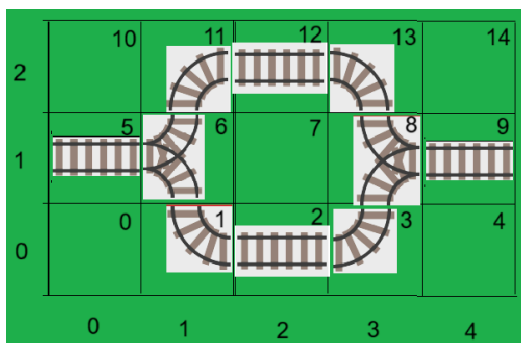
And describe accordingly.

So, there is a full blockage again. What should I do? Let's explore where it may come from.

Let's switch off all the constraints and explore the pure 'path' atoms:

```
path(west,8,north,13) path(west,8,south,3) path(north,6,west,5) path(south,6,west,5)
path(east,6,north,11) path(east,6,south,1) path(south,3,west,2) path(east,3,north,8)
path(south,1,east,2) path(west,1,north,6) path(north,11,east,12) path(west,11,south,6)
path(north,13,west,12) path(east,13,south,8) path(west,2,west,1) path(west,9,west,8)
path(west,12,west,11) path(east,2,east,3) path(east,5,east,6) path(east,12,east,13)
```

Let's explore if the 'path' atoms correctly describe the path given the image and remember that they must account for switches.



Conclusion: all 'path' atoms correctly describe the path.

Intuition: double paths produced by switches cause conflict with constraints;

Next debugging step: let's turn constraints on one by one to see which one (or more) blocks the solution - because the 'path' atoms are correct.

Let's explore an answer while it's still produced:

```
Answer: 236
transition(1,0,9,8) transition(1,1,8,3) transition(1,2,3,2) transition(1,3,2,1)
transition(1,4,1,6) transition(1,5,6,5)
```

This is a completely chaotic movement so far, but at least it:

- Follows the possible transitions only.
- Is not blocked entirely.

Observation: every constraint added significantly (by a factor of 10) reduces the size of the answer set.

The current one gives consecutive, good paths:

```
Answer: 22
transition(1,0,9,8) transition(1,1,8,3) transition(1,2,3,2) transition(1,3,2,1)
transition(1,4,1,6) transition(1,5,6,5)
```

This one is perfectly legit on the network - with only the problem of not arriving at the destination / starting at a wrong place.

```
transition(1,0,13,8) transition(1,1,8,3) transition(1,2,3,2) transition(1,3,2,1)
transition(1,4,1,6) transition(1,5,6,5)
```

This one is very interesting: it uses the loop in the middle instead of travelling from the available ends - they are not indicated, and there is no preference given to them in any form.

Discovery: the problem seems to arise only from the arrival time constraint. Let's see why.

Yes, it's all about the last constraint. Also LLM gives a good advice:

prolog

Copy

Edit

```
:- end_flat(TRAIN, FINAL_POS, ARRIVAL_TIME),
   not_reached(TRAIN, FINAL_POS, ARRIVAL_TIME - 1).

reached(TRAIN, POS, MOMENT) :- transition(TRAIN, MOMENT, _, POS).
```

The bug was not in the constraint, but again in the 'path' encoding - the wrong next cell was checked.

```
Answer: 1
end_flat(1,9,6) transition(1,0,5,6) transition(1,1,6,11) transition(1,2,11,12)
transition(1,3,12,13) transition(1,4,13,8) transition(1,5,8,9)
Answer: 2
end_flat(1,9,6) transition(1,0,5,6) transition(1,1,6,1) transition(1,2,1,2)
transition(1,3,2,3) transition(1,4,3,8) transition(1,5,8,9)
```

Let's set the next goal: to introduce an even more complex environment - the one where choosing a wrong direction on the switch would indeed matter, unlike the last one where *any* path led to the goal.

09.05.25

Where should I go next? I drafted yesterday: implement an environment where choosing a wrong path on the switch would lead to not reaching the goal (we could put a trap there if traps were allowed). I'll draft it on paper this time because I'm on the train. Let's keep it simple - the goal is to test the model of the ability to avoid a bad turn. Let's also name it: 'switch_trap.lp'

Yes, it seems to handle it easily:

```
transition(1,0,3,4) transition(1,1,4,1) transition(1,2,1,2)
```

What next thing I can do?

There are a few of options:

- a. Introduce an even more 'treacherous' environment (don't think it will change anything)
- b. Standardize the output with the task
- c. Introduce speed
- d. Introduce the second train (perhaps, better before speed)

Let's start with a:

Let's use a diamond crossing this time. I will again draw on paper since it seems faster. Again, let's go simple. Just the fact that a diamond crossing is involved is enough. Later, this track will be a good material to test two trains. Let's call it 'basic diamond'.

It also makes sense to test the 'impossible' (right angle) turn on the diamond switch by setting the arrival station at a right angle to the departure (expectation: must return "unsatisfiable"). My assumption now is that it must not even be created - but who knows where a bug hides.

Unsatisfiable.

Why?

H1: I forgot to describe diamond crossing in 'path' atoms creation. +

H2: I described diamond crossing incorrectly in terms of directions in the 'path' atoms creation part.

OBSERVATION: I didn't implement all the track types in 'path' atom creation!

OBSERVATION: paper route prototyping is much faster, stick with it.

Let's retest. Still unsatisfiable. Works.

Bug: in the environment encoding, as it often is - always look there first when it doesn't work.

Let's now shift the destinations to enforce an impossible term. The positive outcome in this case is 'unsatisfiable'. Wrong! The train 'turns' on the diamond cross - why?!

Let's check the 'path' atoms. If there is a turn on the diamond crossing, something is wrong with 'path' creation:

```
path(north,1,north,4) path(south,7,south,4) path(east,3,east,4)
path(west,5,west,4) path(north,4,north,7) path(south,4,south,1)
path(west,4,west,5) path(east,4,east,3)
```

No, all 'path' atoms are correct: there is no turning anywhere, just as it must be. What's wrong then?

Got it: 'path' exists for coupling of two cells, and there is nothing in my encoding that would prohibit such a turn:

The impossible motion:

```
transition(1,0,3,4) transition(1,1,4,1)
```

Happens on legit 'path' atoms:

```
path(east,3,east,4)
path(south,4,south,1)
```

Intuition: shouldn't there also be clenching between entrance / exit directions, not only cell numbers? Let's look at the correct moves:

This correct transition:

```
transition(1,0,3,4) transition(1,1,4,5)
```

Happens on this path:

```
path(east,3,east,4) path(east,4,east,5)
```

So yes, it looks like there is clenching between both direction and position. But let's see what it looks like on a legal turn:

The correct motion of a Γ -turn network:

```
transition(1,0,0,3) transition(1,1,3,6) transition(1,2,6,7)
transition(1,3,7,8)
```

Happens on the path:

```
path(north,0,north,3) path(north,3,north,6) path(north,6,east,7)
path(east,7,east,8)
```

Or is there still clenching? Yes, there is! Then it's still done via a constraint, but it is much clearer to write.

Let's explore clenching between atoms.

No, my assumption was wrong: legit turns look just like 'illegal' ones, and I don't see a simple way to express it.

Then, the obvious way to solve it is through another general (full scope of the model) constraint:

Constraint Idea	ASP Phrasing	Code
Don't let the train make 'illegal' turns.	There must be no answer set where there is a 'transition' atom whose entry and exit positions are not supported by a pair of clenched 'path' atoms. INTUITION: the "TO_CELL" must be right next as well!	<pre>:- transition(_,_,CELL,_), path(_,_,OUT_DIR,CELL), path(IN_DIR,CELL,_,_), OUT_DIR != IN_DIR.</pre>

Example of what must be prohibited:

```
path(_,_,east,4), path(south,4,_,1) => transition(_,_,4,1)
```

'There must be no 'transition' atoms created from two path atoms such that the "to cell" of the first is the "out_cell" of the second, but the "out direction" of the first is not the "out direction" of the second, and also that the "to cell" of the transition atom itself is the same as the 'out' cell of the second 'path' atom.

```
:- transition(_,_,FROM_CELL,TO_CELL),
   path(_,_,OUT_DIR,FROM_CELL),
   path(IN_DIR,FROM_CELL,_,TO_CELL),
   OUT_DIR != IN_DIR.
```

Q: Does different naming of variables in asp imply difference in values?

LLM:

In Answer Set Programming (ASP) — and in particular in Clingo — different variable names do not imply different values. If you want to enforce that two variables are different, you must do so explicitly with an inequality like:

prolog

Copy

Edit

X != Y

I managed to implement this constraint, but it **seems unreliable**. Anyway, let's test how it prohibits other wrong turns on the diamond crossing. It does.

But what if we send the train south-north? My intuition is that it will prohibit this motion altogether.

Let's remove it and set this problem aside for a while - it's minor, but taking up a lot of time.

—

What's the next thing to do?

Let's implement one more complex environment that includes choosing the destination.

Let's call it 'many turns'. It's a 4x4 grid, so we'll need 16 cells:

```
cell((0,0), 0).  
cell((0,1), 32800).  
cell((0,2), 72).  
cell((0,3), 1025).  
cell((1,0), 1025).  
cell((1,1), 34864).  
cell((1,2), 32800).  
cell((1,3), 0).  
cell((2,0), 0).  
cell((2,1), 32872).  
cell((2,2), 5633).  
cell((2,3), 1025).  
cell((3,0), 0).  
cell((3,1), 32800).  
cell((3,2), 0).  
cell((3,3), 0).
```

Let's try different combinations of start and finish, beginning with the simplest one that implies straight track.

Surprisingly (I assumed mistakes in the environment), it works:

```
transition(1,0,13,9) transition(1,1,9,5) transition(1,2,5,1)
```

Let's try a slightly more complicated track - with one turn.

And, surprisingly, it works again:

```
transition(1,0,13,9) transition(1,1,9,10) transition(1,2,10,11)
```

Let's now try the most difficult path that is possible on this route - with two turns, and check if the model will be able to find it:

```
transition(1,0,13,9) transition(1,1,9,10) transition(1,2,10,6)
transition(1,3,6,2) transition(1,4,2,3)
```

Perfect. Everything works. We can keep complicating the environment - where further? I think we can pause at this point and focus on other impending tasks, namely:

- Build a two-train environment, learning to avoid collisions;
- Introduce speed characteristic;
- Standardize the output (not so trivial!).

—

Let's start with a. What is the simplest environment to have 2 trains? It seems that it is a diamond crossing, where two trains follow perpendicular tracks. Let's modify the existing diamond crossing accordingly.

Let's first naively run it without minding any collisions:

```
transition(1,0,3,4) transition(2,0,1,4) transition(1,1,4,5)
transition(2,1,4,7)
```

What is a collision then? How do I express it given how my 'transition' atoms work?

Intuition: let's define a collision as a situation when two trains *enter* the same cell at the same moment of time (we see that they also leave the same cell, expectedly, but it's enough to define by the entry cell).

How do I handle collisions? Two moments:

- There must be a constraint prohibiting two trains to enter the same cell at the same moment of time.
- The time of arrival must be shifted for one of the trains - otherwise it will never arrive.

Let's first deal with the constraint:

Constraint Idea	ASP Phrasing	Code
Two trains must not enter the	There must be no answer set	: -

same cell at the same moment of time.	where there are two 'transition' atoms for different trains, the same moment of time and the same entry cell.	<pre> transition(TRAIN_1, MOMENT, _, TO_CELL), transition(TRAIN_2, MOMENT, _, TO_CELL), TRAIN_1 != TRAIN_2. </pre>
---------------------------------------	---	--

The expected behaviour now is that the constraint will kill all answers. Yes, it happened.

For either of the trains alone, it doesn't prohibit movement - so, the constraint itself is correct.

Now I need to handle the tricky part - **the waiting time**.

First, let's naively increase time for one train and see what happens. Nothing good - it's just unsatisfiable.

INTUITION: what if I first turn 'transition' atoms into 'action' atoms?

CONCERN: my 'transition' atoms in their current state don't imply any waiting;

INTUITION: extend 'transition' atoms with 'no transition':

```
transition(south, 4, south, 4).
```

How do I produce them?

Let's consult with an LLM. Yes, I was on the right track (seemingly).

Let's analyse these answers:

Answer: 1

```
transition(1,0,0,0) transition(1,1,0,3) transition(1,2,3,6)
```

Answer: 2

```
transition(1,0,0,3) transition(1,1,3,6) transition(1,2,6,6)
```

Answer: 3

```
transition(1,0,0,3) transition(1,1,3,3) transition(1,2,3,6)
```

Green encodes movement; blue encoding waiting.

The question is: does my current encoding permit creating many waiting atoms for the same cell?

Let's extend waiting time even further and see what happens:

Answer: 1

```
transition(1,0,0,0) transition(1,1,0,3) transition(1,2,3,3)
transition(1,3,3,6)
```

Answer: 2

```
transition(1,0,0,3) transition(1,1,3,3) transition(1,2,3,6)
transition(1,3,6,6)
```

Answer: 3

```
transition(1,0,0,0) transition(1,1,0,3) transition(1,2,3,6)
transition(1,3,6,6)
```

And more, but with a big step this time:

```
transition(1,0,0,0) transition(1,1,0,3) transition(1,2,3,3)
transition(1,3,3,6) transition(1,4,6,6)
```

ISSUE: longer already doesn't work, because there are no more waiting atoms available - how do I solve this?

Let's set this aside for a while. What if we try two trains already?

Surprisingly, it worked:

```
transition(1,0,3,4) transition(1,1,4,5) transition(1,2,5,5)
transition(1,3,5,4) transition(2,0,1,1) transition(2,1,1,4)
transition(2,2,4,7) transition(2,3,7,7)
```

Let's analyse the answer.

ISSUE: I must make max_time per train - now it's general.

Wow, it worked again:

```
transition(1,0,3,4) transition(1,1,4,5) transition(2,0,1,1)
transition(2,1,1,4) transition(2,2,4,7) transition(2,3,7,7)
```

The output looks very good now. I can jump, while still having some issues, to the task of standardizing the output. Let's look at what it must look like:

action:

- `action(train(ID), move_left, T)`
- `action(train(ID), move_right, T)`
- `action(train(ID), wait, T)`
- `action(train(ID), move_forward, T)`

So, what I'm expected to give is:

```
action(TRAIN, ACTION, MOMENT).
```

Now, the question is how to express 'transition' atoms through 'action' atoms. Let's do it by hand first:

Take the output of diamond crossing solution:

```
transition(1,0,3,4) transition(1,1,4,5) transition(2,0,1,1)
transition(2,1,1,4) transition(2,2,4,7) transition(2,3,7,7)
```

And express in actions:

```
transition(1,0,3,4) => action(1, 0, move_forward)
transition(1,1,4,5) => action(1, 1, move_forward)
transition(2,0,1,1) => action(2, 0, wait)
transition(2,1,1,4) => action(2, 1, move_forward)
transition(2,2,4,7) => action(2, 2, move_forward)
transition(2,3,7,7) => action(2, 3, wait)
```

What code will do this translation for me?

```
action(TRAIN,MOMENT, wait) :- transition(TRAIN,MOMENT,POS_1,POS_2),
POS_1 == POS_2.
```

Let's introduce this alone! Works, let's now attempt translating the movement atoms:

```
action(TRAIN, MOMENT, move_forward) :- transition(TRAIN, MOMENT,
POS_1, POS_2), POS_1 != POS_2.
```

We'll now do it naively, as if switches don't exist.

```
action(1,0,move_forward) action(1,1,move_forward)
action(2,1,move_forward) action(2,2,move_forward) action(2,0,wait)
action(2,3,wait)
```

Looks good.

—

Once this is done, let's **push the fresh changes to git**.

12.05.25

Now I can:

- a. Revise what has been done.
- b. See what I can add before today's session.
- c. Push all updates to git.

I don't remember which track is now used, but let's just run the code and see the output.

Strangely, it returns two identical answers:

```
action(1,0,move_forward) action(1,1,move_forward)
action(1,2,move_forward) action(1,3,move_forward)
action(1,4,move_forward) action(1,5,move_forward)
```

```
action(1,0,move_forward) action(1,1,move_forward)
action(1,2,move_forward) action(1,3,move_forward)
action(1,4,move_forward) action(1,5,move_forward)
```

Why? Let's ask an LLM:

Understanding: because they are produced by different underlying atoms as a generalization for both different sets.

It's definitely a one-train environment. It's 'bagel'. But then isn't the answer just wrong? Looks confusing. Let's start with all simpler environments again and then progress to a more complex one - right from the straight line!

Ah! I see now (after testing on 2 envs): I forgot the format of the 'action' atoms - it's just *train - moment - action*. There's no indication of the position or direction in the action atoms.

```
transition(1,0,3,3) transition(1,1,3,2) transition(1,2,2,2)
```

This is a wrong transition. Why does it occur? Let's try to debug.

Discovery: there is a problem: if there is a cell in the previous row, but with an immediately preceding index - the prohibiting condition of not going off tracks doesn't apply and the train jumps over the whole grid.

How do I solve this?

Idea (looks like makeshift): we could use prohibition to start moving in the direction different from the one indicated in the starting cell.

Constraint Idea	ASP Phrasing	Code
Transition from the starting cell must be only in the direction indicated in the 'start' atom.	There must be no answer set where there is a transition atom (let's see!) that relies on a path atom: <pre>path(west,3,west,2)</pre> And a start atom: <pre>start_flat(1,3,0,east)</pre> With mismatching directions. This is the wrong transition: <pre>transition(1,1,3,2)</pre>	<pre>:- start_flat(_, POSITION,_, DIRECTION_1), path(DIRECTION_2, POSITION,_,_), DIRECTION_1 != DIRECTION_2.</pre>

ISSUE: fix the issue with scalability of the positional converter.

IDEA: it's a good idea to set a higher time limit within which the train still might get late; earlier arrival is not a problem;